

Multi-Model Deployment Algorithms

This document describes three deployment transition strategies implemented for switching between model-device assignments (combinations) in a multi-model inference scheduling system.

Mode 0 : Stop-and-Restart
Mode 1 : Adaptive Hot-Swap
Mode 2 : Reactive (Adaptive + Rollback/Fallback)

Each mode is triggered when a new combination is requested via `update_combination()` in `UnifiedViewer`.

Entry point: `unified_viewer.py :: update_combination()`
if reactive -> `ReactiveDeployManager` (`reactive_deploy.py`)
elif adaptive -> `AdaptiveDeployManager` (`adaptive_deploy.py`)
else -> stop + reinitialize (`unified_viewer.py`)

QoS Violation Score formula:

$$V(t) = V_L(t) + \alpha * D_{\text{tilde}}(t), \quad \alpha = 0.3$$
$$V_L(t) = (1/N) * \text{SUM} \max(0, L_i(t)/L_{\text{SL0},i} - 1)$$
$$D_{\text{tilde}}(t) = D(t) / \min(T, t) \quad (T = 5\text{s window})$$

Mode 0: Stop-and-Restart

File: unified_viewer.py :: update_combination(adaptive=False)

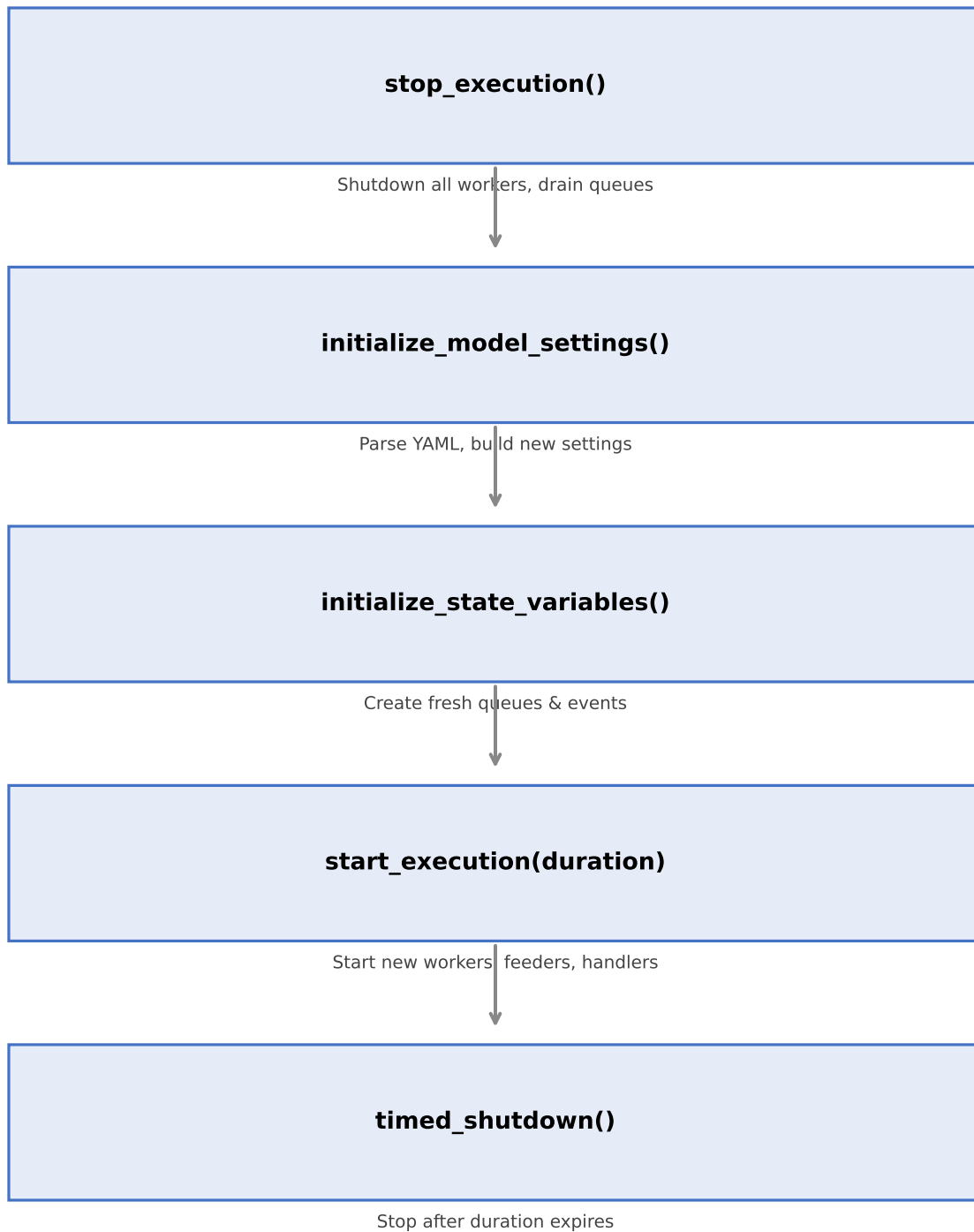
Algorithm:

1. stop_execution()
 - Set shutdown_flag and all per-view shutdown_events
 - Join all worker threads (timeout 3s each)
 - Drain all inter-process queues
 - Save throughput data
2. initialize_model_settings(schedule_file, combination_name)
 - Parse YAML, build model_settings dict
 - Determine views_without_model
3. initialize_state_variables()
 - Create fresh queues, events, shutdown_flag
4. start_execution(duration) [called by executor]
 - initialize_processes(): create worker threads per view
 - initialize_threads(): create feeders + view handlers
 - Schedule timed_shutdown after duration

Characteristics:

- All models stop completely during transition
- Zero throughput for several seconds (model loading time)
- $V(t)$ spikes during the gap
- Simplest implementation, no state carried over

Mode 0: Stop-and-Restart Flow



Mode 1: Adaptive Hot-Swap

File: adaptive_deploy.py :: AdaptiveDeployManager.execute()

Algorithm:

1. Snapshot old model_settings (deep copy)
2. Parse new combination from YAML independently
(does NOT call initialize_model_settings on the viewer)
3. Per-view comparison (view1..view4):
IF same model + same device -> KEEP (no action)
IF device changed -> HOT-SWAP:
 - a. Create new queues (frame_queue, output_queue)
 - b. Create new worker thread with ready_event
 - c. Start new worker (model begins loading)
 - d. Watcher thread waits on ready_event
 - e. On ready:
 - Swap feeder queue (thread-safe lock)
 - Swap handler output queue (atomic under GIL)
 - Signal old worker shutdown
 - Drain old queues
4. Incrementally update viewer state:
 - Only overwrite model_settings for swapped views
 - Kept views retain original handler references + stats
 - Update current_combination label
5. Update feeder view-sets (yolo_views, resnet_views)

Key: model_processors.py run_*_process() functions accept ready_event parameter. After ONNX model loading completes, ready_event.set() is called to signal readiness.

Characteristics:

- Same-device models: zero downtime
- Changed-device models: old worker serves until new ready
- No throughput gap (kept workers continue processing)
- Faster recovery than Mode 0

Mode 1: Adaptive Hot-Swap Flow



Mode 2: Reactive (Rollback / Fallback)

File: reactive_deploy.py :: ReactiveDeployManager.execute()

Extends Mode 1 with post-deployment quality monitoring.

Phase 1: Adaptive hot-swap (delegates to AdaptiveDeployManager)

- Identical to Mode 1

Phase 2: Background monitor thread (5s stabilisation)

- Waits 5 seconds for new deployment to stabilise
- Measures stable $V(t)$ using `_collect_vscore()`

Decision logic after stabilisation:

Case A: Same model set, different combination

IF $V(t)_{\text{new}} - V(t)_{\text{prev}} \geq 5.0$:

- > ROLLBACK to previous combination
(calls `AdaptiveDeployManager.execute()` again)

ELSE: keep new deployment

Case B: Different model set

IF $V(t) > 40.0$:

-> FALLBACK HEURISTIC:

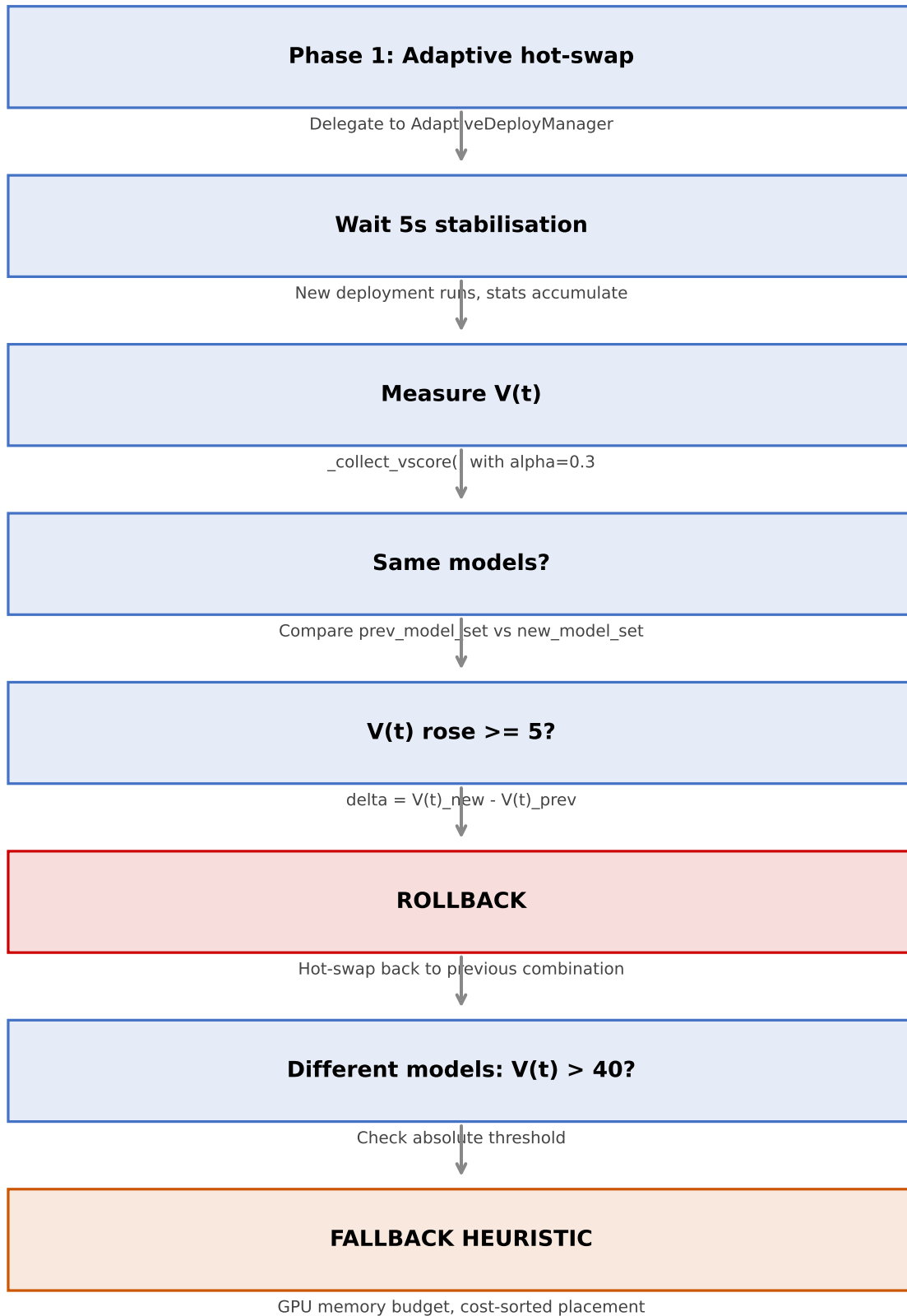
1. Collect per-model inference cost (`avg_infer_time_ms`)
2. Query GPU available memory (`nvidia-smi`)
3. Sort models by cost descending
4. Greedily assign to GPU within memory budget
5. Remaining models -> CPU
6. Write `_reactive_fallback.yaml`
7. Apply via `AdaptiveDeployManager`

ELSE: keep new deployment

Characteristics:

- Self-correcting: reverts bad deployments automatically
- Same-model rollback catches regression within 5 seconds
- Different-model fallback uses runtime cost + GPU memory

Mode 2: Reactive Flow



Algorithm Comparison

	Mode 0 Stop-and-Restart	Mode 1 Adaptive Hot-Swap	Mode 2 Reactive
Transition method	Stop all, restart all	Hot-swap only changed views	Hot-swap + monitor & correct
Downtime	Full stop (seconds)	Zero for kept views	Zero for kept views
Throughput gap	Yes (model loading)	Minimal (old serves)	Minimal (old serves)
Self-correcting	No	No	Yes (rollback/fallback)
Key file	unified_viewer.py	adaptive_deploy.py	reactive_deploy.py
Complexity	Low	Medium	High

Experimental Setup

Objective:

Compare the three deployment transition strategies under a controlled scenario where a GOOD deployment is switched to a WORSE deployment (same models, degraded placement).

Hardware:

- CPU: Multi-core x86_64 (Linux 6.8)
- GPU: NVIDIA GPU with CUDA 12, ONNX Runtime GPU EP

Models (4 concurrent):

- mnasnet (input SL0: 370 FPS)
- resnet50 (input SL0: 73 FPS)
- resnext50 (input SL0: 58 FPS)
- squeezenet1.0 (input SL0: 400 FPS)

Schedule (tests/model_schedules_test.yaml):

combination_1 (GOOD): resnet50 GPU, resnext50 GPU,
mnasnet CPU, squeezenet CPU
combination_2 (WORSE): all 4 models on CPU

Protocol:

1. Run combination_1 for 20s (5s warmup + 15s measured)
2. Transition to combination_2 (same models, all CPU)
3. Run combination_2 for 35s (20s base + 15s extended)
All modes run the same total duration so graphs align.
4. Record per-second metrics: total FPS, per-view latency, drop rate, $V_L(t)$ violation score
5. Full $V(t) = V_L(t) + 0.3 * D_tilde(t)$ computed post-hoc

Execution (scripts/compare_adaptive_modes.py):

- Each mode runs in an independent process (subprocess)
- Process fully terminates before the next mode starts
- `schedule_executor_main.py --auto_start_all --adaptive-mode M`
- Headless rendering (QT_QPA_PLATFORM=offscreen)
- Metrics CSV records 1 row/second after warmup
- Transition artifacts (FPS=0 ticks) are filtered out
- Mode 0/1 duration extended by 15s to match Mode 2's rollback observation window

Mode 2 specifics:

- After transition, monitor thread waits 5s stabilisation
- Measures $V(t)$; if $\delta \geq 5.0$ vs previous, ROLLBACK
- Executor extends run by 15s to observe recovery

X-axis in all graphs: time relative to deploy change ($t=0$).

Negative values = before transition (stable baseline).

Results: QoS Violation Score

$V(t) = V_L(t) + 0.3 * D_tilde(t)$, $\alpha=0.3$, $T=5s$ window

Before transition ($t < 0$):

- All modes run combination_1 (GPU deployment)
- All modes: $V(t) \sim 14$ (low, GPU handles workload well)

At transition ($t = 0$):

- Mode 0 (Stop-and-restart):
All workers stopped. $V(t)$ spikes to ~55 as new CPU workers load ONNX sessions from scratch.
- Mode 1 (Adaptive hot-swap):
Kept workers (mnasnet, squeezenet) continue on CPU.
Changed workers (resnet50, resnext50: GPU->CPU) are hot-swapped. $V(t)$ rises gradually (~14 -> ~24 over 35 seconds). See 'Cumulative Average Dilution' page.
- Mode 2 (Reactive):
Same initial transition as Mode 1. After 5s, monitor detects $V(t)$ increase ≥ 5.0 . Triggers ROLLBACK.
 $V(t)$ recovers to ~15.

After stabilisation ($t > 10$):

- Mode 0: $V(t)$ stabilises at ~43 (all CPU, high latency)
- Mode 1: $V(t)$ slowly rises to ~24 (see next page for why)
- Mode 2: $V(t)$ returns to ~15 (rolled back to GPU)

Analysis: Mode 0 V-Score Oscillation (t=0 to t=5)

Observation:

Mode 0's V(t) oscillates between t=0 and t=5 after the deploy change, despite no further deployment changes.

Raw data (combination_2, all CPU):

t=0: V_L=41.82, FPS=19.01 (first inference after restart)
t=1: V_L=45.22, FPS=18.49 (peak)
t=2: V_L=44.11, FPS=18.66 (descending)
t=3: V_L=43.03, FPS=19.20
t=4: V_L=42.68, FPS=19.37
t=5: V_L=39.38, FPS=20.79 (dip)
t=6: V_L=41.88, FPS=20.46 (back up)

Root cause 1: ONNX Runtime cold-start effect (t=0 to t=4)

After stop-and-restart, all 4 models create new ONNX InferenceSession objects. The first inferences trigger:

- JIT graph optimisation within ONNX Runtime
- CPU instruction cache misses (fresh process memory)
- Memory allocation for intermediate tensors

avg_infer_time is a cumulative moving average:

avg = total_infer_time / infer_count

With few samples (t=0~1), cold inferences dominate the average -> V_L peaks at ~45. As warm inferences accumulate (t=2~4), the average descends gradually.

Root cause 2: Measurement window reset at t=5

start_execution() schedules _begin_measurement_window() at t+5s. This calls handler.reset_stats() on all views:

total_infer_time = 0.0
infer_count = 0
avg_infer_time = 0.0

After reset, the first few inferences are measured in a warm state (CPU caches hot, ONNX session optimised). This produces a temporarily lower avg_infer_time -> V_L dips to ~39. As 4 concurrent CPU models compete for resources, contention raises latency back to ~42-47.

Conclusion:

The oscillation is a measurement artifact caused by cold-start latency and cumulative average reset, NOT a change in model deployment. The deployment remains fixed at combination_2 (all CPU) throughout.

Analysis: Why Mode 1 V-Score << Mode 0

Observation:

Mode 1's $V(t)$ stabilises at ~20-24, while Mode 0 reaches ~43. Both run combination_2 (all CPU) with no rollback. Mode 1 should converge to Mode 0 but does not.

Root cause: Cumulative average dilution

Mode 1's view handlers are NOT recreated during hot-swap. The handler's `avg_infer_time` is a cumulative average:

$$\text{avg} = \text{total_infer_time} / \text{infer_count}$$

For a swapped view (e.g., resnet50: GPU -> CPU):

combination_1 (GPU, 15s at ~250 FPS):

`infer_count` ~ 3750, `total_infer` ~ 15000 ms (4ms each)

combination_2 (CPU, 15s at ~5 FPS):

`infer_count` ~ 75, `total_infer` ~ 12750 ms (170ms each)

Combined cumulative average:

$$\text{avg} = (15000 + 12750) / (3750 + 75) = 7.25 \text{ ms}$$

V_L contribution for this view:

$$L_{SL0} = 1000/73 = 13.7 \text{ ms}$$

Mode 1: $\max(0, 7.25/13.7 - 1) = 0$ (no violation!)

Mode 0: $\max(0, 170/13.7 - 1) = 11.4$ (severe violation)

The 3750 fast GPU samples dilute the 75 slow CPU samples, making `avg_infer_time` artificially low.

Why no stats reset in Mode 1:

Mode 0 calls `start_execution()` for combination_2, which triggers `_begin_measurement_window()` -> `reset_stats()` at +5s. Mode 1 skips `start_execution()` (viewer already `_run_active`), so `reset_stats()` is never called.

Convergence:

Mode 1 will eventually converge to Mode 0's V-Score, but it requires the CPU sample count to overwhelm the GPU history. At 5 FPS, this takes ~750s (12.5 minutes) to equal the 3750 GPU samples.

Implication:

Mode 1's lower V-Score is partially a measurement artifact. The actual inference latency is identical to Mode 0. Mode 2's advantage is genuine: it rolls back to GPU.

Results: Latency and Drop Rate

Total Inference Latency (sum of per-view avg_infer_time_ms):

Before transition:

All modes: ~150 ms total (GPU models have low latency)

After transition:

- Mode 0: Jumps to ~800 ms (4 models all on CPU)
Stays high for the entire remaining duration.
- Mode 1: Gradual increase as GPU->CPU swap completes.
Rises to ~200-250 ms and continues increasing
as cumulative CPU contention grows.
- Mode 2: Brief rise to ~200 ms during combination_2,
then drops back to ~170 ms after rollback
restores GPU deployment.

Drop Rate (frames dropped per second due to full queues):

Before transition:

All modes: ~80-100 FPS (normal operating drop rate
due to input rate exceeding inference throughput).

After transition:

- Mode 0: Drops to near 0 during restart (no input
fed), then recovers to ~80-100 FPS.
- Mode 1: Maintained at ~80-90 FPS (continuous feeding,
gradual increase as CPU workers are slower).
- Mode 2: Similar to Mode 1 initially, then stabilises
at ~90 FPS after rollback restores throughput.

Results: Summary and Key Findings

Quantitative comparison (latest run, equal duration):

Metric	Mode 0	Mode 1	Mode 2

Samples	68	65	50
Avg Throughput (FPS)	351	415	432
Avg V-Score	31.31	17.59	15.42
Max V-Score	54.94	23.44	17.43
Avg Drop Rate (FPS)	115.5	115.0	112.1

Key findings:

1. Mode 0 (Stop-and-restart) has the highest V-Score (31.31) due to the complete throughput gap during model reloading and sustained high-latency CPU execution for the entire post-transition period.
2. Mode 1 (Adaptive) shows lower V-Score (17.59) but this is PARTIALLY a measurement artifact: cumulative avg_infer_time carries fast GPU history that dilutes the slower CPU latency (see Dilution Analysis page). The actual inference latency equals Mode 0's. Mode 1's V(t) slowly rises toward Mode 0's level but convergence takes ~12 minutes.
3. Mode 2 (Reactive) achieves the lowest V-Score (15.42, 51% lower than Mode 0). Unlike Mode 1, this advantage is GENUINE: Mode 2 detects the degradation within 5 seconds and rolls back to the GPU deployment, restoring actual low latency.
4. Mode 2's rollback is visible in the time-series as a brief V(t) increase followed by recovery. The 'Rollback' vertical line marks self-correction.
5. Mode 2 achieves the highest throughput (432 FPS) because it spends most time on the optimal GPU deployment rather than the degraded CPU one.